

IMAGE BLURRING REPORT

Team Number: 8

Team Members

Student ID	Section	Student Name
4	1	اية محمد سيد
8	1	حنين مصطفى محمد عطا
36	2	نادية كامل محمد علي

Table of Content

1. Project Overview

2. Project Architecture

a. How files are organized - The Flow

3. Network Setup:

a. Two-subnet configuration.

4. How to Run

a. Single image vs. Batch processing

5. Comparison Between input and output

6. Final Performance Report

7. Conclusion

1. Project Overview

This project demonstrates a high-performance distributed image processing cluster built using OpenMPI and Docker. The primary objective is to implement a scalable system capable of performing intensive spatial stencil operations (specifically, a Gaussian/Box Blur) on massive image matrices exceeding 100 million pixels (10,000 x 10,000).

Key Objectives:

- **Distributed Computation:** Utilizing the Master-Worker paradigm to partition large-scale image data and process it concurrently across multiple nodes.
- **Environment Virtualization:** Leveraging Docker to create a consistent, reproducible, and isolated high-performance computing (HPC) environment.
- **Network Simulation:** Implementing a complex dual-subnet network architecture to simulate inter-node communication and routing in a distributed data center environment.
- **Scalability:** Demonstrating how increasing the number of MPI worker nodes and "slots" significantly reduces the total execution time compared to traditional serial processing.

2. Project Architecture

The system follows a Master-Worker design pattern, orchestrated within a containerized environment to ensure high availability and resource isolation.

Key Components:

- **Master Node (Orchestrator):**
 - Acts as the central gateway for the cluster.
 - **Handles Image I/O:** Reads the source image from the shared volume and prepares it for distribution.
 - **Workload Partitioning:** Divides the image into horizontal strips (chunks) to be distributed among workers.
 - **Aggregation:** Collects processed data from all workers and reconstructs the final image.

- **Worker Nodes (Computational Units):**
 - Consists of four identical Docker containers (worker1 to worker4).
 - Each worker operates as an independent MPI process.
 - Executes the core stencil operation (Convolution kernel) on its assigned data segment.
- **Communication Layer:**
 - OpenMPI: Used for message passing and synchronization between nodes.
 - SSH (Secure Shell): Configured for passwordless communication between containers, allowing the MPI runtime to launch processes across the cluster seamlessly.
 - Shared Volumes: Used to store the final output images, accessible by the host machine for verification.

How Files are Organized - The Flow

The project maintains a structured directory to ensure modularity and ease of deployment.

Directory Structure:

- /docker: Contains the Dockerfile and setup scripts to build the environment.
- /src: The core C++ source code, including main.cpp (orchestration), mpi_manager.cpp (communication), and blur_processor.cpp (algorithm logic).
- /images: Directory for input test images (HD and 10K resolutions).
- /output: Target folder for the final processed images and performance logs.
- docker-compose.yml: The configuration file that defines services, networks, and volumes for the cluster.
- hostfile: A configuration file for MPI that specifies node IP addresses and available CPU slots.

The Execution Flow:

- **Deployment:** docker-compose builds the shared image and initializes the five containers (Master + 4 Workers).

- **Compilation:** The Master node uses CMake to compile the source code into an executable binary within the shared environment.
- **Data Partitioning:** The Master node loads the image using OpenCV and partitions it into horizontal segments (strips) based on the number of workers.
- **Distribution (Scatter):** Using MPI_Scatterv, the Master sends the respective image chunks and the convolution kernel to all worker nodes.
- **Parallel Computation:** Each worker performs the stencil operation concurrently on its segment.
- **Aggregation (Gather):** The Master collects the processed segments via MPI_Gatherv.
- **Finalization:** The Master reconstructs the image, applies final normalization, and saves the result to the output volume.

3. Network Setup

To simulate a complex enterprise-grade cluster, we implemented a Multi-Subnet Network Architecture rather than a flat network.

Network Segmentation:

- **Subnet A (172.25.0.0/24):** Contains the Master node (172.25.0.2) and Workers 1 & 2.
- **Subnet B (172.26.0.0/24):** Contains the Master node (172.26.0.2) and Workers 3 & 4.
- **The Master as a Router (Gateway):** The Master node is configured as a Multi-Homed Host, meaning it is physically connected to both subnets. We enabled IP Forwarding on the Master node, allowing it to act as a bridge/router that routes MPI traffic between the two isolated subnets.

Static Routing & Communication:

- Since workers in Subnet A do not have a direct route to Subnet B, we manually injected Static Routes into the worker containers.
- Each worker in Subnet A is configured to send any traffic destined for the 172.26 range through the Master's IP (172.25.0.2).
- This setup tests the robustness of the MPI communication layer, ensuring that the TCP BTL (Byte Transport Layer) in OpenMPI can navigate across different network hops using the Master as a gateway.

4.How to Run

The project is designed for ease of deployment and testing through a two-step process using Docker and automated shell scripts.

1. Initializing the Cluster: To spin up the virtual nodes and establish the dual-subnet network, use the following command from the project root:

`docker-compose up -d --build`

This command ensures that the environment is consistent and that the Master and four Workers are connected according to the network topology.

2. Executing the Processing Pipeline: The system supports two main modes of execution through a dedicated Bash script located on the Master node:

- Single Image Mode: Used for rapid testing and verification. It processes a single image (HD resolution) and generates a side-by-side comparison.

`./run_mpi_blur.sh --image images/test_hd.jpg --np 10`

- Batch Processing (Performance Mode): Automates the execution across multiple images of varying sizes. This mode is used to gather timing data for the final performance report.

`./run_mpi_blur.sh --batch`

3. Advanced MPI Parameters: The execution script internally handles the complex OpenMPI configuration required for the multi-subnet environment, including:

- `--allow-run-as-root`: Necessary for Docker execution.
- `--mca btl_tcp_if_exclude`: Used to force MPI traffic through the specific 172.x subnets, ignoring default Docker interfaces.

5.Comparison Between input and output



6.Final Performance Report

```
=====
FINAL PERFORMANCE REPORT
=====
Image Name                | Avg Time (sec)
-----
input_1.jpg               | 121.952744
input_2.jpg               | 119.925148
input_4.jpg               | 121.113761
input_3.png               | 111.024824
input_5.png               | 114.380443
-----
Grand Total Average for 5 images: 117.6793 seconds
=====
```

This is Final report about 5 i/p images and their avg time and finally totalavg time

7. Conculsion

This project successfully demonstrates the implementation of a robust, containerized distributed system for high-performance image processing. The primary achievements include:

1. **Scalability:** The Master-Worker model proved effective in partitioning large-scale data, showing that parallelization is essential for handling 10K+ resolution images.
2. **Network Resilience:** We successfully navigated the complexities of inter-subnet communication. By configuring static routes and enabling IP forwarding on the Master node, we simulated a real-world multi-network cluster environment.
3. **Portability:** Utilizing Docker ensured that the entire High-Performance Computing (HPC) stack—including OpenCV, OpenMPI, and SSH configurations—is portable and can be deployed on any host with identical results.
4. **Mathematical Integrity:** Through rigorous gathering and reconstruction of image chunks, the system maintained 100% data integrity, producing seamless filtered outputs without artifacts at the chunk boundaries.

In summary, the combination of OpenMPI for communication and Docker for orchestration provides a powerful and cost-effective framework for developing and testing complex distributed algorithms before deploying them to physical supercomputing clusters.

Team Participation

Code , Repo , Report	Member
33%	Nadia Kamel
33%	Hanin Mustafa
33%	Aya Mohamed